
What Counts as a Mistake? Annotating Recitation Events in Quran Memorization Transcripts

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Checking Quran recitation from an ASR transcript requires distinguishing unre-
2 solved mistakes from repetitions, repairs, opening formulas and accepted spelling
3 differences. We report a completed human annotation of 100 production record-
4 ing cases: 348 scored units and 162 localized events across ten combined la-
5 bels. An executable evaluator scores labels and word positions together. A plain
6 diff reaches label-aware F1 0.525 and localization F1 0.826; adapted production
7 cleaner/alignment components reach 0.518 and 0.786, with exact-span F1 0.505
8 for both. Correcting the adapter’s word coordinates recovers all five annotated
9 repetition events, showing why annotation interfaces must be checked before inter-
10 preting baseline failures. In a preliminary pilot, seven single 20-minute runs across
11 three coding agents and seven models span label-aware F1 0.143 to 0.892: six land
12 far above every baseline, and one collapses below the naive diff from a missing
13 normalization step, an informative failure as much as the successes are. The dataset
14 and scorecard support the repeated-run agent comparison this pilot precedes.

15 1 The distinction the task turns on

16 Quran memorization is normally practised with a listener who corrects the reciter. An application
17 that stands in for that listener transcribes with commercial ASR and compares the transcript against
18 the reference. Edit distance locates differences but does not determine their interpretation. In our
19 production data 74% of reviewed ayah units carry no event at all, and among those that do, a
20 substantial share are things a human listener would never mention: the reciter repeated a phrase after
21 a breath, said a word wrongly and immediately fixed it, or used a written form differing from the
22 Uthmani orthography without differing in what was recited. Flagging these as unresolved mistakes
23 can give the learner misleading feedback; we do not measure the relative user cost here.

24 We therefore annotate transcript-against-reference differences as typed, localized *events*, and we
25 judge the text rather than the speaker: a label records what differs between transcript and reference,
26 not whether the reciter or the ASR caused it. Unwritten vowel and tajweed errors lie outside a
27 transcript-only rubric and outside this work.

28 **Related work.** Machine support for Quran recitation has mostly worked from audio: verse delimita-
29 tion with speech recognition [Tabbal et al., 2006], tajweed-rule classification from the acoustic signal
30 [Alagrami and Eljazzar, 2020], and pretrained encoders for Quranic ASR [Hossain et al., 2026],
31 reviewed by Balula et al. [2021]. The datasets follow the same shape: Tarteel crowd-sources and
32 labels recitations [Khan et al., 2021], a later corpus labels non-Arabic speakers’ recitations in six
33 categories with inter-rater agreement [Salameh et al., 2024], and Quran-MD aligns verse and word
34 audio with text [Salman et al., 2025]. All mark a recording or a phone correct or incorrect; we label

the transcript, typing and locating every difference. Pronunciation assessment scores a learner attempt against a reference [Witt and Young, 2000], and disfluency annotation records a repetition or a repair as its own structure rather than as an error [Shriberg, 1994]; our rubric applies the second idea to an ASR transcript against a fixed reference. The coding agents we score are normally measured on software and ML engineering benchmarks [Jimenez et al., 2024, Chan et al., 2025].

2 Taxonomy and annotation

Combined labels. An earlier pilot used two parallel fields, an event type and a benign/mistake verdict, plus an uncertain escape hatch. Reviewing real recordings made both untenable: the verdict was never independent of the type, and uncertain absorbed exactly the cases that needed a decision. The rubric now uses one combined label per event, `substitution_mistake`, `omission_mistake`, `insertion_mistake`, `substitution_corrected`, `omission_corrected`, `repetition_benign`, `letters_benign`, `spelling_benign`, `basmala_benign` and `isti3adha_benign`, with `clean` for a reviewed unit carrying no event. A unit reviewed and found clean is not the same as a unit not yet reviewed, and the data distinguishes them.

Four decisions that took adjudication. *Omissions* include missing beginnings, interiors and endings, with no exemption for the final ayah of a recording: stopping early still leaves reference material absent from the transcript, and a reviewer cannot infer intent from recording position. *Repeats* span the original phrase together with its extra occurrence against a single reference copy, aligning their localization with corrected events; a wrong-then-correct attempt is a corrected event, while a correct-then-incorrect restatement is a `substitution_mistake` spanning both attempts, with the order recorded in the note. *Spelling* differences removed by an agreed context-independent normalization receive no event, and only a residual form needing contextual acceptance receives `spelling_benign`; the accepted contexts are enumerated rather than generalized, so the category cannot quietly become blanket final-letter deletion. *Localization* uses verbatim words and half-open spans into the original whitespace token arrays, with an empty hypothesis span anchoring an omission and an empty reference span anchoring an insertion.

Scale and composition. All 100 selected recording cases are human-approved: 314 ayah units, 233 of them clean, 115 within-ayah events and 47 opening-formula annotations, over 274 ayahs in 38 surahs. The event distribution is uneven by nature, from 62 `substitution_mistake` down to one `insertion_mistake` and one `omission_corrected`, and any evaluation has to survive that. Ten learner identities contribute, one of them 46 recordings. Selection was purposive, so these counts describe the annotated set; they do not estimate error prevalence or imply generalization to unseen reciters. Three further approved recordings remain outside the scored set; their transcript-only copies join the development candidates, with annotations archived privately. We verified identifiers and transcripts against both source exports and excluded punctuation-normalized duplicates and redundant clean subpassages. Gold answers and selected membership remain in an owner-held private bundle; this draft does not release a gold sample.

3 An executable scorecard

Systems receive the reviewed ayah split, the ayah ids and the exact references, so detection and segmentation are supplied rather than scored; that task is a companion submission. Per unit a system returns events of the form `{label, hyp_span, ref_span}`, with half-open spans into the transcript and reference token arrays.

Opening formulas are scored. Each record’s *isti’adhah* and *basmala* are presented as their own unit, with the formula as the transcript and an empty reference; a compound opening yields two events over one token array. Deciding that a *basmala* preceding the ayah is benign rather than an insertion is the judgment under test; excluding it would hide the failure mode that matters most.

Matching. Predictions and gold events are paired one-to-one within a unit by maximum total similarity, so a single broad prediction is credited with at most one gold event and the rest count as misses. A pair is eligible only when *both* spans overlap, $\min(\text{sim}_{hyp}, \text{sim}_{ref}) \geq 0.50$; averaging the two, as an earlier version did, lets an exact reference span carry a match while the hypothesis span points elsewhere. Events are validated for known labels, integer endpoints, token bounds and

label-specific empty spans. Invalid predictions count as false positives. Empty spans match only empty anchors, exactly or within one token; they never match selected words. Pairing ignores labels so localization can be reported separately. Units with more than seven events on either side use deterministic greedy matching.

Scores. The primary measure is label-aware event F1: a paired prediction is a true positive only when its label also matches, and a mislabeled pair counts once as a false positive and once as a false negative. We report micro F1 as the headline, exact-span F1 requiring identical endpoints, macro F1 across labels present in gold, and localization precision/recall/F1 with labels dropped. We also report an application-facing cost in raw event counts, (r missed + false flags) per 100 units, at $r = 1$ and $r = 2$, because no exchange rate between a missed mistake and an unnecessary flag is established. Neither rate ranks systems and the ordering is unchanged from 1:1 to 5:1; the rate matters only in that predicting nothing wins below roughly 0.77:1. Predicting nothing gives F1 0 and the gold annotation gives 1. Evaluator v2.1 passes synthetic coordinate, label, anchor and adapter regression tests and private gold oracle checks. Sweeping the tolerance from 0.05 to 1.00 moves baseline micro F1 by about four points and leaves exact-span F1 unchanged, since diff-based baselines emit exact spans; 0.50 is chosen to be explainable rather than tuned. One-token anchor slack changes no baseline score across 0 to 10 and is retained because the index of an omission between two tokens is genuinely ambiguous. A system predicting looser spans will depend on the tolerance more than these baselines.

4 What existing systems do

System	micro F1	exact F1	macro F1	loc F1	review cost
Predict nothing	0.000	0.000	0.000	0.000	51.2
Plain diff	0.525	0.505	0.178	0.826	23.3
Adapted production components	0.518	0.505	0.284	0.786	26.7
Codex, GPT-6-Astra (pilot)	0.892	0.862	0.701	0.934	8.1
Claude Code, Opus 5 (pilot)	0.880	0.849	0.660	0.931	9.2
Codex, GPT-5.6-Sol (pilot)	0.875	0.863	0.631	0.938	9.5
Claude Code, Sonnet 5 (pilot)	0.856	0.825	0.609	0.912	9.8
Antigravity, Gemini 3.6 Flash (pilot)	0.797	0.773	0.387	0.878	14.1
Antigravity, Gemini 3.1 Pro (pilot)	0.795	0.677	0.502	0.873	10.9
Antigravity, gpt-oss-120B (pilot)	0.143	0.121	0.017	0.187	169.3

Table 1: Evaluator v2.1: 348 units, 162 gold events. No run crashes; the gpt-oss-120B run produces 22 invalid events, discussed below. The seven pilot rows are one 20-minute development run per agent/model pair, not the preregistered comparison of Section 5.

The plain diff locates 126 of 162 gold events (recall 0.778), with localization F1 0.826. Its label-aware recall is 0.494 and F1 is 0.525. These quantities differ: localization F1 is not the percentage of gold events found. It reaches per-label F1 0.94 for omissions and 0.85 for substitutions, and zero for the other eight labels. It emits 46 insertion predictions, 34 of them on opening-text units, whereas gold has one insertion event. Compound openings contain two gold formulas but a diff can return one insertion spanning both.

The second baseline adapts production cleaner and alignment components to the new event schema; the deployed application does not natively expose it. The adapter retains original token identities through deletion, uses alignment indices and reference offsets, and maps cleaner notes to reference phrases and both attempts. Its heuristic interprets a stumble as a repair only when the following equally long surviving span matches the reference; unlocalizable removed fragments remain extra text. Production’s partial-start scoring policy is retained. These choices are part of the baseline, not additional gold decisions.

This adapter predicts five repetitions and recovers all five (F1 1.0); six corrected-substitution predictions recover one of four gold events (F1 0.20). It has nonzero F1 on four labels and zero on six, and never predicts either opening-formula label, letters, contextual spelling or omission corrections. The earlier adapter lost valid repetition credit through incorrect coordinates. The remaining label gaps motivate explicit event modeling, but they do not establish that cleaning causes the errors, or measure every feature of the deployed application. Each baseline retains its own comparison normalization, an algorithm choice rather than a change to the gold rubric.

126 **A preliminary agent pilot.** Each of three coding agents (Codex, Antigravity, Claude Code) was
127 given the unlabelled 127-recording training release, the annotation guide and 20 constructed teaching
128 examples with answers, a 20-minute development budget, and no access to gold or the internet; each
129 froze one `solution.py` that we ran, unmodified, over the private gold set. Codex ran under GPT-
130 5.6-Sol and GPT-6-Astra; Antigravity under Gemini 3.1 Pro, Gemini 3.6 Flash and gpt-oss-120B;
131 Claude Code under Opus 5 and Sonnet 5, once each. Seven runs, seven models, one run per model:
132 this is not the repeated-run comparison of Section 5, no model, budget or seed was frozen in advance,
133 and nothing here ranks agents or models in general.

134 Six of the seven reach micro F1 0.79–0.89, far above every baseline, with zero invalid predictions
135 across all 348 units. GPT-6-Astra is highest overall (0.892) and on macro F1 (0.701), narrowly ahead
136 of Claude Code under Opus 5 (0.880) and Codex under GPT-5.6-Sol (0.875, highest on localization at
137 0.938); Claude Code under Sonnet 5 (0.856) and the two Antigravity/Gemini runs (0.797, 0.795) com-
138 plete the six. All three agents reach per-label F1 1.00 on `basmala_benign` and `isti3adha_benign`
139 in at least one run; `spelling_benign` is the clearest shared weakness, F1 0.15–0.67 across all
140 six, and the two single-example labels (`insertion_mistake`, `omission_corrected`) are where
141 every run is weakest, unsurprisingly given one teaching example each. Gemini 3.1 Pro recov-
142 ers `isti3adha_benign` perfectly but only 5 of 18 `basmala_benign` events, consistent with an
143 opening-formula handler returning one label per unit where a compound opening carries two.

144 The seventh run is an informative failure. gpt-oss-120B’s solution never normalizes Arabic orthogra-
145 phy before diffing, so ordinary hamza and alif-form differences register as `substitution_mistake`
146 on nearly every token (545 of 567 predicted events carry that one label); it also treats every opening-
147 formula unit as clean, missing all 47 `basmala_benign/isti3adha_benign` events, and produces
148 22 invalid events from a schema mismatch on its insertion/omission spans. The result, micro F1 0.143
149 and a 68% clean-flag rate, is worse than either non-agent baseline: the task’s difficulty is not diffing,
150 and a diff-shaped answer under the same instructions, guide and examples as the six successful runs
151 can still fail on the comparison step that Section 2’s rubric assumes is solved.

152 5 Limitations and what comes next

153 The annotated set is purposive and small, one learner contributes 46 of the 100 recordings, and the
154 label distribution is heavily skewed, so macro F1 here is a coverage check, not a stable estimate.
155 More consequentially, all 100 selected transcripts already appear as inputs in a public release of the
156 same recordings, and twelve in an earlier machine-labeled pilot. The adjudications and the selection
157 membership are private, but the inputs are not, and runtime isolation cannot establish absence of
158 prior exposure; an unseen-input claim requires a separate corpus. The prepared development release
159 contains 127 cases and 888 units, including 44 of 63 newer recordings. It permits shared clean ayahs
160 and different error variants, but excludes complete gold copies and event-bearing gold chunks. The
161 one local ASR repair appears non-Quranic and is excluded. Remaining new recordings are not a
162 frozen hidden test.

163 We plan to compare agents under the same unlabelled training data, rubric, examples, tools and
164 budget. Agents may annotate, train models or write rules; annotation is an available strategy, not the
165 experimental contrast. The pilot above is a first, informal instance of this design, run to check that the
166 harness and isolation work end to end before the preregistered protocol is fixed. Isolation differed
167 by runtime. Codex and Claude Code each ran non-interactively in a fresh git-free directory outside
168 this repository, under their sandbox’s default network restriction, with an isolated credentials-only
169 home directory carrying no prior session history, cache or goals, one fresh instance per model.
170 Antigravity ran in a fresh window and directory per model; its network access was not independently
171 instrumented, and its selected model is taken on the operator’s report rather than verified externally.
172 All seven saw only the unlabelled release, the guide and the teaching examples, never gold, and
173 froze their solution before any gold scoring. A high score supports the combined process, not each
174 training label: code may ignore or correct its labels. Generated annotations need independent review,
175 but a fully human-labelled training pool is unnecessary. Gold answers, selected test bundles and
176 review history stay outside development environments; shared clean inputs and prior public exposure
177 are disclosed. The repeated-run comparison, with budgets, models and annotation delivery fixed in
178 advance, remains future work.

References

- Ali M Alagrami and Maged M Eljazzar. Smartajweed automatic recognition of arabic quranic recitation rules. *arXiv preprint arXiv:2101.04200*, 2020.
- Nazik O’mar Balula, Mohsen Rashwan, and Shrief Abdou. Automatic speech recognition (asr) systems for learning arabic language and al-quran recitation: a review. *International Journal of Computer Science and Mobile Computing*, 10(7):91–100, 2021.
- Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, et al. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *International Conference on Learning Representations*, volume 2025, pages 50466–50494, 2025.
- Nabil Mosharrarf Hossain, Riasat Islam, and Unaizah Obaidellah. A comparative study of pretrained transformer models for quranic asr: Speech representations, label formats, and dataset composition. *arXiv preprint arXiv:2606.19747*, 2026.
- Carlos E Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. Swe-bench: Can language models resolve real-world github issues? In *International Conference on Learning Representations*, volume 2024, pages 54107–54157, 2024.
- Hamzah I Khan, Abubakar Abid, Mohamed Medhat Moussa, and Anas Abou-Allaban. The tarteel dataset: crowd-sourced and labeled quranic recitation. 2021.
- Raghad Salameh, Mohamad Al Mdfaa, Nursultan Askarbekuly, and Manuel Mazzara. Quranic audio dataset: Crowdsourced and labeled recitation from non-arabic speakers. *Procedia Computer Science*, 246:2684–2693, 2024.
- Muhammad Umar Salman, Mohammad Areeb Qazi, and Mohammed Talha Alam. Quran-MD: A fine-grained multimodal dataset of the quran. In *5th Muslims in ML Workshop co-located with NeurIPS 2025*, 2025.
- Elizabeth Ellen Shriberg. Preliminaries to a theory of speech disfluencies. *Doctoral dissertation, University of California at Berkeley*, 1994.
- Hassan Tabbal, W El Falou, and B Monla. Analysis and implementation of a" quranic" verses delimitation system in audio files using speech recognition techniques. In *2006 2nd international conference on information & communication technologies*, volume 2, pages 2979–2984. IEEE, 2006.
- Silke M Witt and Steve J Young. Phone-level pronunciation scoring and assessment for interactive language learning. *Speech communication*, 30(2-3):95–108, 2000.